

Tập 0: Setup Lab Environment

Kubernetes Networking — 45 tập thực hành

Cluster chạy local · Ubuntu 26.04 · Kubernetes 1.36 · Multipass

Tổng quan lab

1 cluster Kubernetes · 3 node · chạy trên máy tính của bạn

macOS / Windows Host (kubectl, helm, multipass CLI)

controlplane
control-plane
2 CPU · 2 GB RAM

worker1
worker
1 CPU · 1.5 GB

worker2
worker
1 CPU · 1.5 GB

Ubuntu 26.04 LTS · containerd · kubelet

Multipass = hypervisor nhẹ, chạy VM Ubuntu thật trên macOS/Windows — không cần Docker Desktop hay cloud account.

Yêu cầu máy host

Resource	Tối thiểu	Khuyến nghị
RAM	8 GB	16 GB
CPU	4 core	8 core
Disk	60 GB free	100 GB free
OS	macOS 13+ / Windows 10+	macOS M-series

```
# Cài Multipass (macOS)
brew install multipass
```

```
# Verify
multipass version
# multipass 1.16.x
```

Cấu trúc lab files

```
tap-00-setup-lab/
├── k8s-cloud-init.yaml  ← cloud-init: "bản thiết kế" chung mỗi VM
├── setup-lab.sh         ← Router tự động nhận diện CPU & điều hướng
├── setup-lab-arm.sh     ← Dành riêng cho chip ARM (Apple Silicon)
├── setup-lab-amd.sh     ← Dành riêng cho chip AMD/Intel (x86_64)
└── reset-lab.sh        ← Xóa toàn bộ VMs sạch sẽ
```

k8s-cloud-init.yaml — tự động hóa hoàn toàn nhờ `dpkg --print-architecture`:

- Tắt swap, load kernel modules (`overlay` , `br_netfilter`)
- Cài `containerd` + `kubelet` , `kubeadm` , `kubectrl` v1.36 phù hợp chính xác với CPU của bạn.

Bước 1 — Tạo máy ảo bằng Script phù hợp

Hệ thống hỗ trợ 2 cách chạy cực kỳ linh hoạt:

Cách 1: Chạy Router tự động (Khuyến dùng)

```
cd tap-00-setup-lab/  
./setup-lab.sh          # Tự phát hiện CPU & gọi script tối ưu
```

Cách 2: Chạy trực tiếp tùy theo chip của máy bạn

```
./setup-lab-arm.sh      # Nếu dùng máy Mac M1/M2/M3/M4 (ARM)  
# HOẶC  
./setup-lab-amd.sh      # Nếu dùng Windows/Linux hoặc Mac Intel (AMD/Intel)
```


Bước 2 — Khởi tạo Control Plane

Truy cập vào controlplane và chạy lệnh khởi tạo:

```
# Shell vào node controlplane
multipass shell controlplane

# Khởi tạo K8s
sudo kubeadm init --apiserver-advertise-address=<IP_CỦA_CONTROLPLANE> --pod-network-cidr=10.244.0.0/16

# Copy cấu hình kubeconfig
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Verify
kubectl get nodes
```

# NAME	STATUS	ROLES	AGE	VERSION
# controlplane	NotReady	control-plane	3m	v1.36.x

NotReady là đúng — cluster chưa có CNI. Cài CNI theo tập học, nodes sẽ chuyển **Ready**.

Bước 3 — Join Worker Nodes vào Cụm

Chạy lệnh `kubeadm join` (được in ra ở cuối Bước 2) trên các worker:

```
# Trên Worker 1
multipass shell worker1
sudo kubeadm join <IP_CỦA_CONTROLPLANE>:6443 \
  --token <token> --discovery-token-ca-cert-hash sha256:<hash>

# Trên Worker 2
multipass shell worker2
sudo kubeadm join <IP_CỦA_CONTROLPLANE>:6443 \
  --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Quay lại controlplane chạy `kubectll get nodes` sẽ thấy 3 nodes (NotReady).

Bước 4 — Cài CNI theo tập học

```
# Flannel – đơn giản, VXLAN overlay (Tập 6–10)
kubectl apply -f \
  https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml

# Calico – BGP, Network Policy (Tập 11–26)
curl -s https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml | \
  sed "s|192.168.0.0/16|10.244.0.0/16|g" | kubectl apply -f -

# Cilium – eBPF, Hubble observability (Tập 27–43)
helm repo add cilium https://helm.cilium.io/
helm install cilium cilium/cilium \
  --namespace kube-system \
  --set hubble.enabled=true \
  --set hubble.relay.enabled=true
```

```
# Sau khi cài CNI → nodes chuyển Ready
kubectl get nodes
```

# NAME	STATUS	ROLES	AGE	VERSION
# controlplane	Ready	control-plane	5m	v1.36.x
# worker1	Ready	<none>	3m	v1.36.x
# worker2	Ready	<none>	3m	v1.36.x

Làm việc hàng ngày với lab

```
# Xem trạng thái cluster
kubectl get nodes -o wide
kubectl get pods -A

# Shell vào node
multipass shell controlplane
multipass shell worker1

# Ví dụ: chạy lệnh sau khi đã vào node
crictl pods
ip route show

# Thông tin IP các node
multipass list
# controlplane      Running  192.168.64.10
# worker1           Running  192.168.64.11
# worker2           Running  192.168.64.12
```

Đổi CNI giữa các module

```
# Xóa toàn bộ VMs
./reset-lab.sh

# Khởi tạo lại VMs mới (rất nhanh vì image Ubuntu đã được cache)
./setup-lab.sh

# Sau đó thực hiện lại Bước 2 (kubeadm init/join)
# Rồi tiến hành cài CNI mới
kubectl apply -f <CNI-manifest>
```

Troubleshooting

```
# Truy cập shell của node (ví dụ controlplane)
multipass shell controlplane

# Kiểm tra cloud-init còn chạy không
cloud-init status
# status: running → chờ thêm
# status: done → OK
# status: error → xem log bên dưới

# Xem log cloud-init
sudo tail -50 /var/log/cloud-init-output.log

# containerd không chạy
systemctl status containerd
sudo systemctl restart containerd

# Nếu lỗi sai sót quá nặng → reset làm lại
./reset-lab.sh
```

Xóa lab và tạo lại

```
# Xóa toàn bộ VMs
./reset-lab.sh

# Tạo lại từ đầu (nhanh hơn lần đầu vì Ubuntu image đã cached)
./setup-lab.sh
```

Cloud-init image 26.04 được Multipass cache sau lần tải đầu tiên — recreate cluster chỉ mất ~2 phút thay vì 5 phút.

Cluster sẵn sàng

Một cluster · dùng cho toàn bộ 45 tập · đổi CNI bằng `reset-lab.sh`

```
./setup-lab.sh    ← chạy một lần duy nhất  
kubectl get nodes ← kiểm tra
```